

Administración Remota de Servidores Web Múltiples en AWS

Rodrigo María Gómez

Fecha: 17/11/2025

0. PREPARACIÓN DEL ENTORNO LOCAL (WSL)

0.1 Crear directorio para las claves SSH

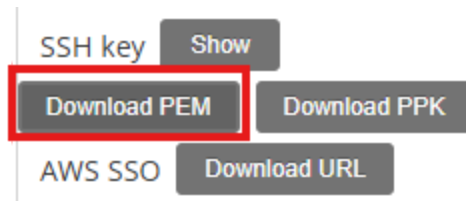
```
rodrigo@A6Alumno11:/mnt/c/WINDOWS/system32$ mkdir -p ~/.ssh  
rodrigo@A6Alumno11:/mnt/c/WINDOWS/system32$ chmod 700 ~/.ssh
```

Descripción: Crea el directorio .ssh con permisos correctos para almacenar claves.

1. CONFIGURACIÓN EN AWS (Interfaz Visual)

1.1 Descargar la clave PEM desde la página del laboratorio

1. Localiza el botón o enlace “Download key” o “Download .pem”
2. Se descargará un archivo con extensión .pem
3. Importante: Guarda este archivo en una ubicación segura, preferiblemente en WSL



1.2 Pasos para mover la clave a WSL:

Descargué el archivo en Windows y lo moví a mi unidad C: para acortar la ruta:

```
cp /mnt/c/NubeProject4.pem ~/.ssh/
```

```
root@A6Alumno11:/mnt/c/WINDOWS/system32# cp /mnt/c/NubeProject4.pem ~/.ssh/
```

Configurar permisos de la clave PEM

```
root@A6Alumno11:/mnt/c/WINDOWS/system32# chmod 400 ~/.ssh/NubeProject4.pem
```

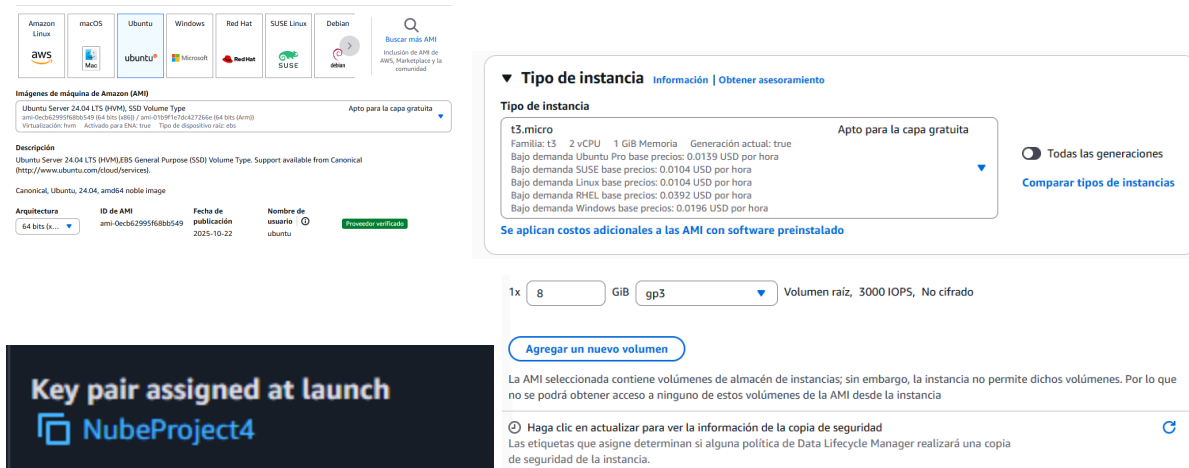
Verifica los permisos: ls -la ~/.ssh/NubeProject4.pem

```
root@A6Alumno11:/mnt/c/WINDOWS/system32# cp /mnt/c/NubeProject4.pem ~/.ssh/  
root@A6Alumno11:/mnt/c/WINDOWS/system32# ls -la ~/.ssh/NubeProject4.pem  
-rwxr-xr-x 1 root root 1674 Nov 18 10:15 /root/.ssh/NubeProject4.pem
```

2. PREPARACIÓN DEL ENTORNO Y SEGURIDAD (AWS)

2.1. Creación de la Instancia EC2

Para realizar esta práctica, hemos accedido a la consola de AWS y seleccionado el servicio EC2. El proceso de creación de la instancia consta de varios pasos críticos para asegurar su correcto funcionamiento y accesibilidad.



2.2. Configuración de Seguridad y Claves

¿Qué es una clave PEM?

Es un archivo que contiene una clave privada criptográfica. Funciona como una llave de seguridad única: AWS guarda la clave pública (el candado) y yo guardo el archivo .pem (la llave). Al conectarnos, se verifica matemáticamente que ambas coinciden sin necesidad de enviar contraseñas por la red, lo que lo hace mucho más seguro

Security Group:

El Security Group actúa como un firewall virtual a nivel de instancia. Hemos configurado una política de "Deny All" por defecto, abriendo explícitamente solo los puertos necesarios: 22 para administración (SSH) y 8080, 8081, 8082, 8443 para los servicios web que desplegaremos.

```
root@A6Alumno11:/mnt/c/WINDOWS/system32# ls -l ~/.ssh/NubeProject4.pem
-r----- 1 root root 1674 Nov 18 10:15 /root/.ssh/NubeProject4.pem
root@A6Alumno11:/mnt/c/WINDOWS/system32# ls -la ~/.ssh/
total 20
drwx----- 2 root root 4096 Nov 18 10:18 .
drwx----- 3 root root 4096 Nov 18 14:42 ..
-r----- 1 root root 1674 Nov 18 10:15 NubeProject4.pem
-rw----- 1 root root 1120 Nov 19 08:45 known_hosts
-rw-r--r-- 1 root root 142 Nov 18 10:14 known_hosts.old
```

Fig 1. Verificación de permisos 400 en el archivo de clave privada situado en el directorio seguro ~/.ssh/ de WSL. Se confirma que solo el propietario tiene acceso de lectura.



Fig 2. Instancia EC2 desplegada en AWS con sistema operativo Ubuntu Server. Se observa la dirección IP pública asignada para el acceso remoto.

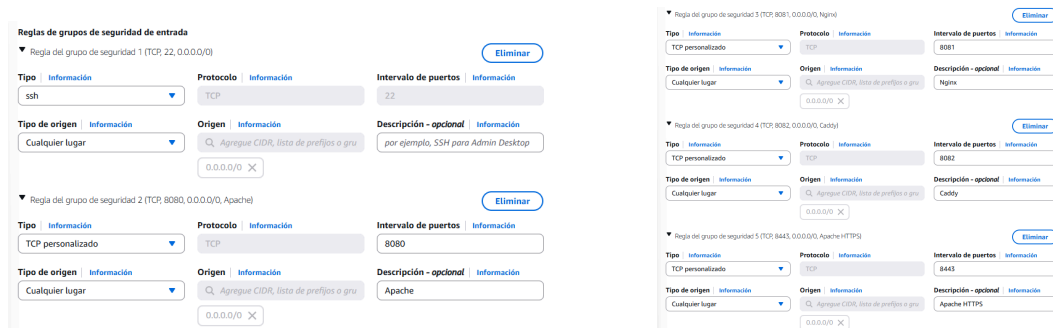


Fig 3. Configuración del Security Group grupo-servidores-web. Se han habilitado reglas de entrada TCP para los puertos personalizados de los tres servidores web y el acceso SSH.

3. CONEXIÓN REMOTA SSH

Objetivo: Establecer un túnel seguro cifrado para la administración del servidor.

El protocolo SSH (Secure Shell) nos permite administrar el servidor remoto de forma segura. Utilizando la clave privada (.pem) previamente configurada, establecemos una conexión cifrada desde nuestra máquina local hacia la IP pública de la instancia en AWS.

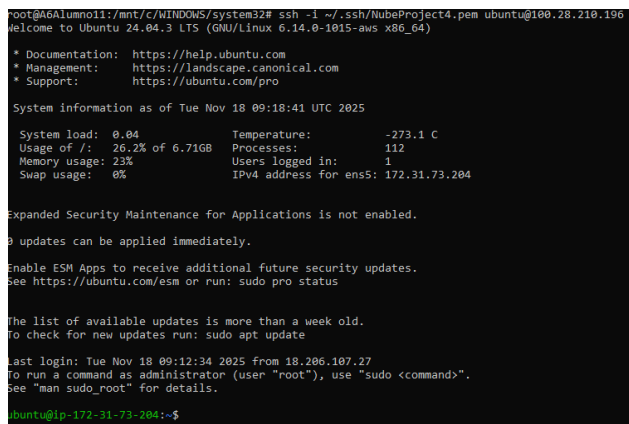


Fig 4. Establecimiento exitoso de la sesión SSH desde la estación local (WSL) hacia la instancia remota EC2 utilizando la clave privada para la autenticación.

4. DESPLIEGUE DE APACHE Y PHP (Puerto 8080)

Configurar el servidor Apache para operar en un puerto no estándar y servir contenido dinámico. (Ya está instalado de prácticas anteriores)

Apache HTTP Server es el servidor web más utilizado del mundo. En esta práctica, hemos modificado su configuración por defecto (que escucha en el puerto 80) para que escuche en el puerto **8080**. No basta con abrir el puerto; debemos decir al sitio por defecto que responda en él. Editamos el archivo de configuración del sitio. Además, hemos instalado el módulo PHP para permitir que el servidor procese contenido dinámico, no sólo estático.

```
ubuntu@ip-172-31-73-204:~$ sudo systemctl status apache2
* apache2.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/apache2.service; enabled; preset: enabled)
   Active: active (running) since Thu 2025-11-20 07:51:53 UTC; 1h 11min ago
     Docs: https://httpd.apache.org/docs/2.4/
   Process: 514 ExecStart=/usr/sbin/apachectl start (code=exited, status=0/SUCCESS)
   Process: 887 ExecReload=/usr/sbin/apachectl graceful (code=exited, status=0/SUCCESS)
   Main PID: 563 (apache2)
    Tasks: 8 (limit: 1017)
   Memory: 29.1M (peak: 36.4M)
      CPU: 575ms
   CGroup: /system.slice/apache2.service
           └─ 563 /usr/sbin/apache2 -k start
           └─ 933 /usr/sbin/apache2 -k start
           └─ 934 /usr/sbin/apache2 -k start
           └─ 935 /usr/sbin/apache2 -k start
           └─ 936 /usr/sbin/apache2 -k start
           └─ 937 /usr/sbin/apache2 -k start
           └─ 1237 /usr/sbin/apache2 -k start
           └─ 1238 /usr/sbin/apache2 -k start
```

Fig 5. El servicio apache2 se encuentra activo y en ejecución (active/running).

```
root@ip-172-31-73-204:/home/ubuntu# sudo sed -i 's/Listen 80/Listen 8080/' /etc/apache2/ports.conf
```

Fig 6. Modificación del archivo ports.conf estableciendo la directiva Listen 8080 para evitar conflictos con puertos privilegiados. Usé el comando **sed** para no tener que abrir el editor.

```
root@ip-172-31-73-204:/home/ubuntu
GNU nano 7.2 /etc/apache2/sites-available/000-default.conf
<VirtualHost *:8080>
    # The ServerName directive sets the request scheme, hostname and port that
    # the server uses to identify itself. This is used when creating
    # redirection URLs. In the context of virtual hosts, the ServerName
    # specifies what hostname must appear in the request's Host: header to
    # match this virtual host. For the default virtual host (this file) this
    # value is not decisive as it is used as a last resort host regardless.
    # However, you must set it for any further virtual host explicitly.
    #ServerName www.example.com

    ServerAdmin webmaster@localhost
    DocumentRoot /var/www/html

    # Available loglevels: trace8, ..., trace1, debug, info, notice, warn,
    # error, crit, alert, emerg.
    # It is also possible to configure the loglevel for particular
    # modules, e.g.
    #LogLevel info ssl:warn

    ErrorLog ${APACHE_LOG_DIR}/error.log
    CustomLog ${APACHE_LOG_DIR}/access.log combined

    # For most configuration files from conf-available/, which are
    # enabled or disabled at a global level, it is possible to
    # include a line for only one particular virtual host. For example the
    # following line enables the CGI configuration for this host only
    # after it has been globally disabled with "a2disconf".
    #Include conf-available/serve-cgi-bin.conf
</VirtualHost>
```

Fig 7. Configuración del VirtualHost en Apache para que el sitio web sea accesible a través del puerto 8080 en lugar del puerto 80 por defecto

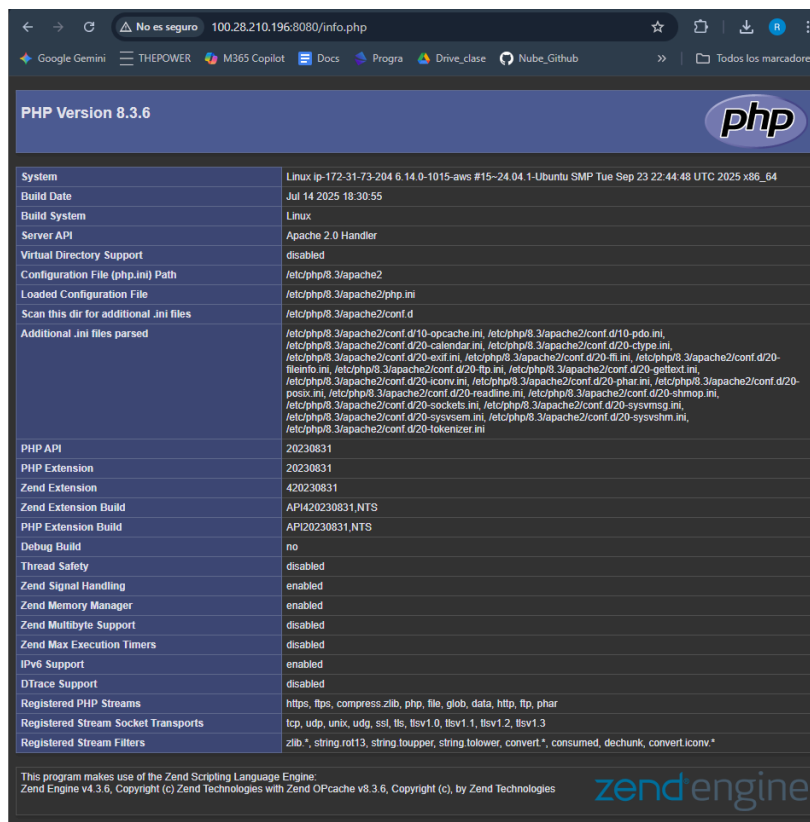


Fig 8. Acceso vía navegador al puerto 8080. Apache procesa correctamente el código PHP mediante el módulo libapache2-mod-php y devuelve la información del sistema.

5. DESPLIEGUE DE NGINX (Puerto 8081)

Nginx es conocido por su alto rendimiento y bajo consumo de recursos. Para que coexista con Apache, lo hemos configurado en el puerto **8081**. Es crucial cambiar también el directorio raíz (root) para que no sirva los mismos archivos que Apache, creando un entorno aislado para este segundo servidor.

```
root@ip-172-31-73-204:/home/ubuntu# sudo mkdir -p /var/www/nginx_custom
root@ip-172-31-73-204:/home/ubuntu# echo "<h1>¡POR FIN! Soy Nginx</h1><p>Funcionando en puerto 8081 y en mi propia carpeta.</p>" | sudo tee /var/www/nginx_custom/index.html
<h1>¡POR FIN! Soy Nginx</h1><p>Funcionando en puerto 8081 y en mi propia carpeta.</p>
root@ip-172-31-73-204:/home/ubuntu# sudo bash -c 'cat > /etc/nginx/sites-available/default <<EOF
server {
    listen 8081 default_server;
    listen [::]:8081 default_server;

    # AQUÍ ESTÁ EL CAMBIO IMPORTANTE:
    root /var/www/nginx_custom;

    index index.html;

    server_name _;

    location / {
        try_files $uri $uri/ =404;
    }
}
EOF'
root@ip-172-31-73-204:/home/ubuntu# sudo systemctl restart nginx
```

Fig 9. Archivo de configuración de Nginx modificado. Se observa la escucha en el puerto 8081 y la ruta personalizada /var/www/nginx_custom para evitar conflictos de archivos con Apache.

```
ubuntu@ip-172-31-73-204:~$ sudo netstat -tulpn | grep nginx
tcp        0      0 0.0.0.0:8081          0.0.0.0:*            LISTEN      631/nginx: master p
tcp6       0      0 :::8081              :::*                  LISTEN      631/nginx: master p
```

Fig 10. Verificación de sockets de red. El proceso Nginx está escuchando correctamente en el puerto TCP 8081.

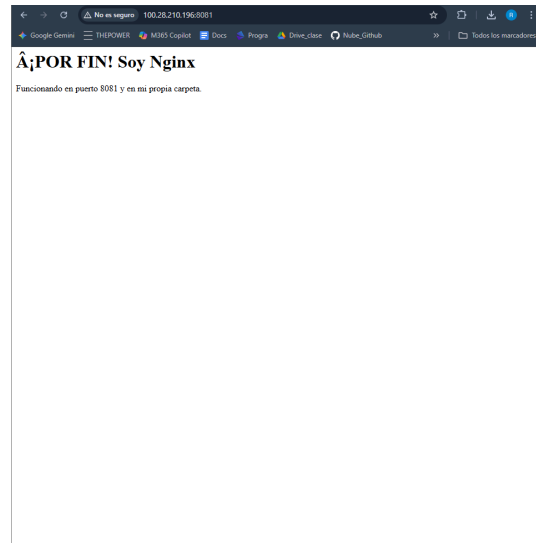


Fig 11. Nginx sirviendo contenido estático HTML personalizado desde su directorio independiente.

6. DESPLIEGUE DE CADDY (Puerto 8082)

Caddy es un servidor web moderno escrito en Go, famoso por su simplicidad y HTTPS automático. En este caso, lo usamos en el puerto **8082** para demostrar su capacidad de `file_server`, lo que permite navegar por los archivos de un directorio visualmente desde el navegador, útil para repositorios de documentos.

```
ubuntu@ip-172-31-73-204:~$ cat /etc/caddy/Caddyfile
:8082 {
    root * /var/www/caddy
    file_server browse
}
```

Fig 12. Declaración del Caddyfile definiendo el puerto 8082 y habilitando la directiva `file_server browse` para permitir la navegación de archivos.

```
ubuntu@ip-172-31-73-204:~$ sudo systemctl status caddy
caddy.service - Caddy
Loaded: loaded (/usr/lib/systemd/system/caddy.service; enabled; preset: enabled)
Active: active (running) since Thu 2025-11-20 07:51:55 UTC; 1h 26min ago
Docs: https://caddyserver.com/docs/
Main PID: 516 (caddy)
Tasks: 8 (limit: 1017)
Memory: 48.9M (peak: 50.1M)
CPU: 347ms
CGroup: /system.slice/caddy.service
         └─516 /usr/bin/caddy run --environ --config /etc/caddy/Caddyfile
```

Fig 13. Caddy ejecutándose correctamente tras la recarga de la configuración.

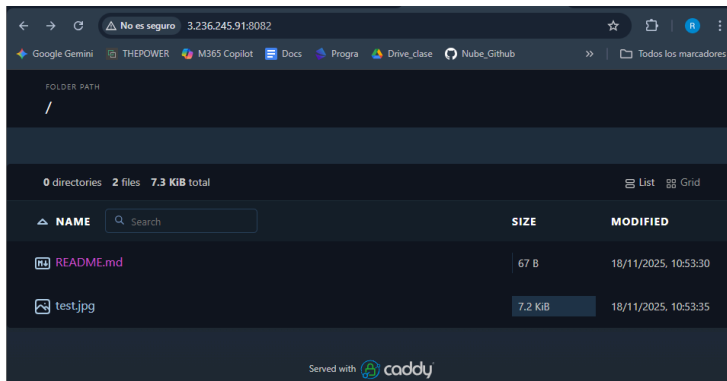


Fig 14. Interfaz web de Caddy mostrando el listado del directorio /var/www/caddy.

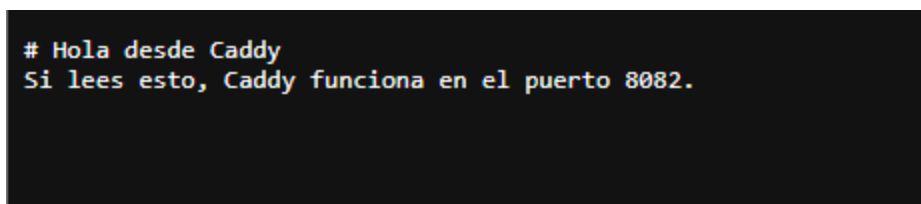
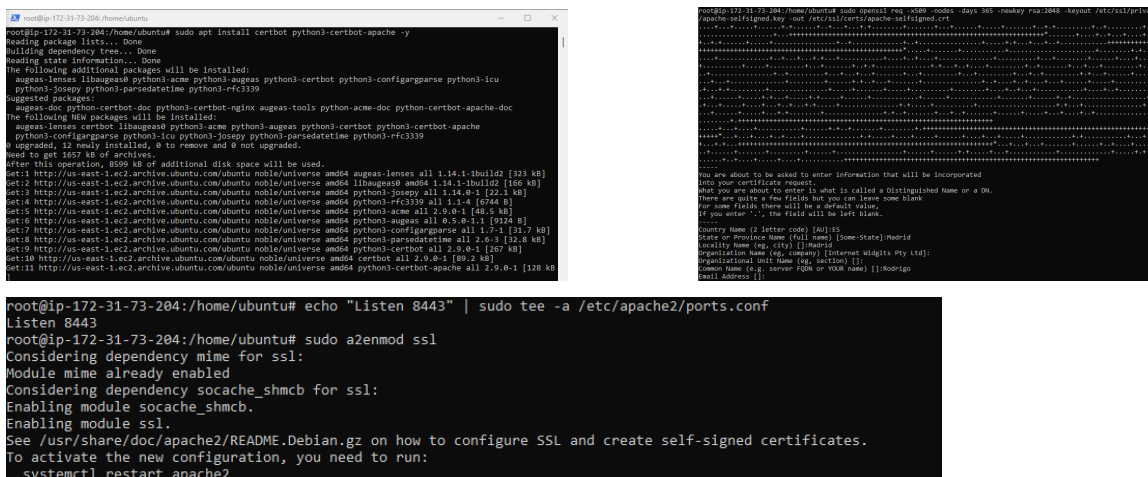


Fig 15. Caddy sirviendo y mostrando el contenido del archivo README.md plano.

7. SEGURIDAD HTTPS EN APACHE (Puerto 8443)

Para asegurar las comunicaciones, hemos habilitado HTTPS en Apache. Dado que no tenemos un dominio real, usamos un certificado "autofirmado" generado con OpenSSL. Configuramos Apache para escuchar tráfico seguro en el puerto **8443** (alternativo al 443 estándar).

******:** Apache ya está instalado y dejo aquí en pequeño la instalación de Certbot y su integración con Apache para gestionar certificados SSL. También he creado un certificado autofirmado para practicar HTTPS localmente completando los campos solicitados.



```

root@ip-172-31-73-204:/home/ubuntu# sudo bash -c 'cat > /etc/apache2/ports.conf <<EOF
# Archivo de configuración de puertos LIMPIO

# Puerto para HTTP normal (Práctica Parte 1)
Listen 8080

# Puertos para HTTPS seguro (Práctica Parte 4)
<IfModule ssl_module>
    Listen 8443
</IfModule>

<IfModule mod_gnutls.c>
    Listen 8443
</IfModule>
EOF'
root@ip-172-31-73-204:/home/ubuntu# sudo apache2ctl configtest
Syntax OK

ubuntu@ip-172-31-73-204:~$ sudo netstat -tulnp | grep apache2
tcp6      0      0 :::8443          :::*              LISTEN    563/apache2
tcp6      0      0 :::8080          :::*              LISTEN    563/apache2

```

Fig 16. Apache escuchando simultáneamente en el puerto HTTP (8080) y el puerto seguro HTTPS (8443).

The figure consists of three screenshots. The top-left screenshot shows the Apache2 Default Page in a web browser, displaying the Ubuntu logo and a 'It works!' message. The top-right screenshot shows the 'Visor de certificados: Rodrigo' window, displaying details of a certificate issued to Rodrigo, including the organization 'Internet Widgits Pty Ltd' and the validity period from November 18, 2025, to November 18, 2026. The bottom screenshot shows a dialog box for generating a certificate, with fields for 'Emisor', 'Validar', 'Posterior a', 'Anterior a', 'Tema', and 'Información de clave pública de la entidad receptora'. The 'Valor de campo' section shows the certificate details: CN = Rodrigo, O = Internet Widgits Pty Ltd, L = Madrid, ST = Madrid, C = ES.

Fig 17. Conexión cifrada establecida a través del puerto 8443. El navegador identifica el protocolo HTTPS, utilizando el certificado SSL autofirmado generado mediante OpenSSL.

8. VERIFICACIÓN FINAL SIMULTÁNEA

Esta sección valida el éxito de la práctica. Hemos logrado que tres tecnologías diferentes (Apache, Nginx, Caddy) compartan los recursos de una única máquina EC2 sin conflictos, gracias a la correcta gestión de puertos y ficheros de configuración.

```
tcp6      0      0 :::8443          :::*              LISTEN      563/apache2
tcp6      0      0 :::8081          :::*              LISTEN      631/nginx: master p
tcp6      0      0 :::8080          :::*              LISTEN      563/apache2
tcp6      0      0 :::8082          :::*              LISTEN      516/caddy
```

Fig 18. Vista consolidada de la tabla de red del sistema. Se evidencia que Apache, Nginx y Caddy están activos simultáneamente, cada uno ocupando su puerto asignado sin generar conflictos de recursos.

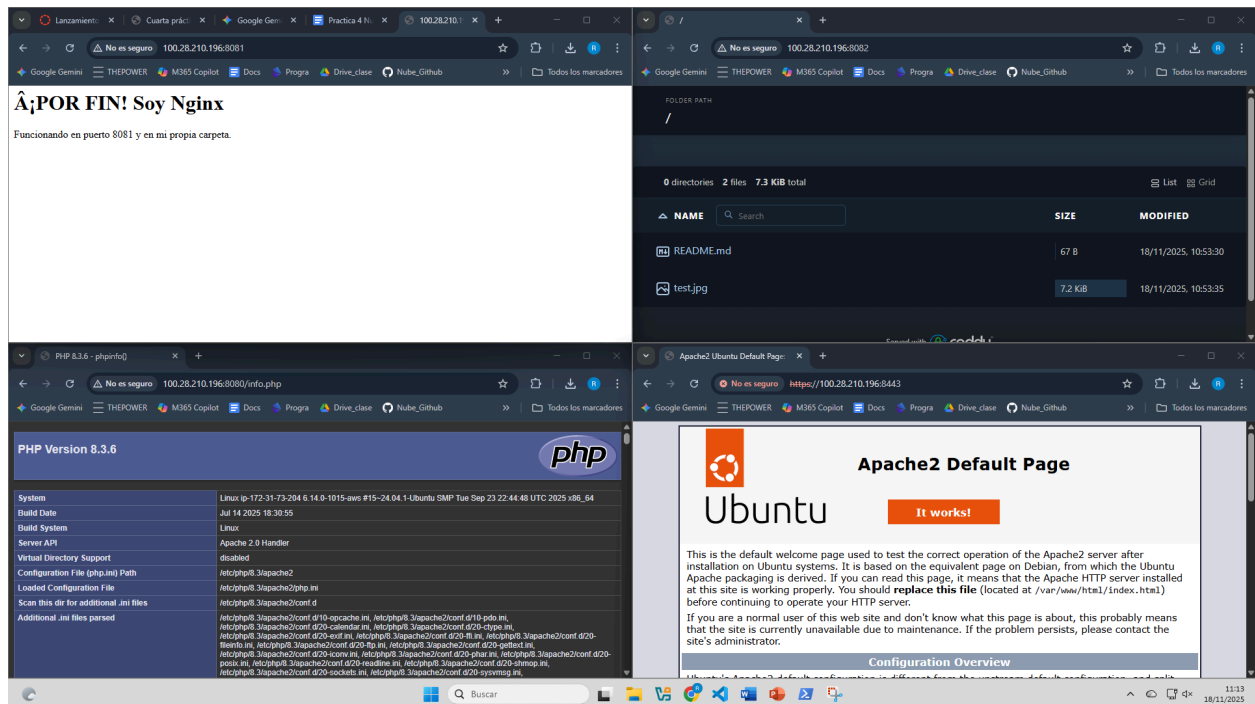


Fig 19. Verificación funcional completa. Los cuatro servicios (Apache PHP, Nginx HTML, Caddy FileServer y Apache HTTPS) responden correctamente de forma simultánea desde la misma dirección IP pública, completando satisfactoriamente la práctica de arquitectura multicapa.